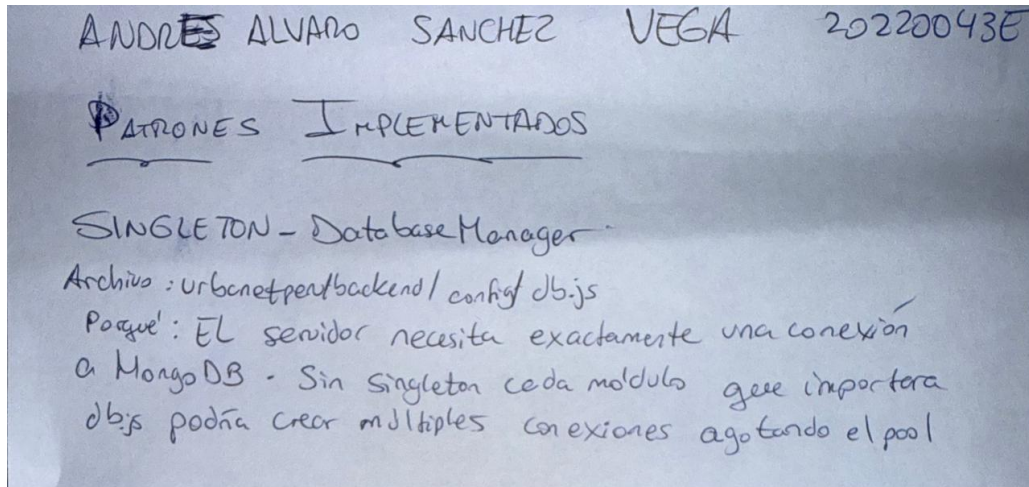


UNIVERSIDAD NACIONAL DE INGENIERÍA
FACULTAD DE CIENCIAS
ESCUELA PROFESIONAL DE CIENCIA DE LA
COMPUTACIÓN

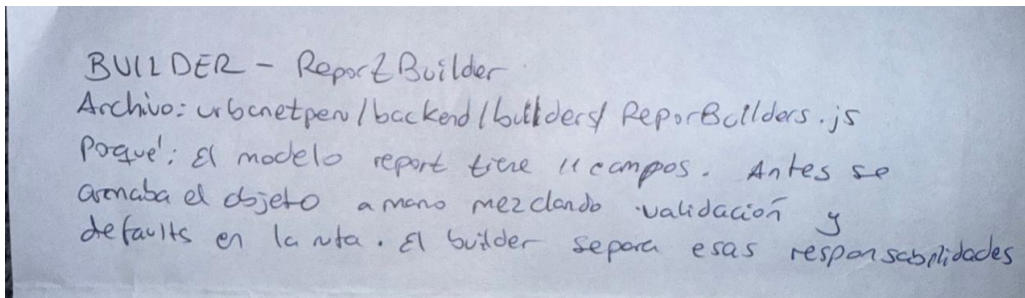
Práctica 2

Curso:	Desarrollo de Software
Profesor:	Quispe Torres Manuel Alejandro
Apellidos y Nombre:	Sánchez Vega Andre (20220043E)
Fecha de entrega:	22/05/2026



Listing 1: backend/config/db.js – Singleton DatabaseManager

```
1 class DatabaseManager {
2   constructor() {
3     this._connected = false;
4   }
5
6   static getInstance() {
7     if (!DatabaseManager._instance) {
8       DatabaseManager._instance = new DatabaseManager();
9     }
10    return DatabaseManager._instance; // siempre el mismo objeto
11  }
12
13  async connect() {
14    if (this._connected) {
15      console.log('MongoDB: reusing existing connection');
16      return;
17    }
18    const conn = await mongoose.connect(process.env.MONGO_URI);
19    this._connected = true;
20    console.log('MongoDB connected: ${conn.connection.host}');
21  }
22
23  isConnected() {
24    return this._connected;
25  }
26 }
```



Listing 2: backend/builders/ReportBuilder.js – Builder

```
1 class ReportBuilder {
2   constructor(userId, userName, userEmail) {
3     this._data = {
4       userId, userName, userEmail,
5       category: 'bache',
6       status: 'reportado',
7       priority: 'media',
8       images: []
9     };
10  }
11
12  setTitle(title) { this._data.title = title; return this; }
13  setDescription(desc) { this._data.description = desc; return this; }
14  setLocation(loc, lat, lng) {
15    this._data.location = loc;
16    if (lat !== undefined) this._data.latitude = lat;
17    if (lng !== undefined) this._data.longitude = lng;
18    return this;
19  }
20  setCategory(cat) { this._data.category = cat || 'bache'; return this; }
21  setPriority(pri) { this._data.priority = pri || 'media'; return this; }
22  setImages(imgs) { this._data.images = imgs || []; return this; }
23
24  build() {
25    if (!this._data.title || !this._data.description || !this._data.location) {
26      throw new Error('Titulo, descripcion y ubicacion son obligatorios');
27    }
28    return { ...this._data };
29  }
30 }
```

Se usó en `backend/routes/reports.js`:

Listing 3: `backend/routes/reports.js` – uso del Builder

```
1 const reportData = new ReportBuilder(  
2   req.user.userId, req.body.userName, req.body.userEmail)  
3   .setTitle(title)  
4   .setDescription(description)  
5   .setLocation(location, latitude, longitude)  
6   .setCategory(category)  
7   .setPriority(priority)  
8   .setImages(images)  
9   .build();  
10  
11 const report = await Report.create(reportData);
```

Cambios realizados:

- **Modificado:** `backend/config/db.js` — se agregó la clase `DatabaseManager` con el patrón Singleton y se actualizó la exportación.
- **Nuevo:** `backend/builders/ReportBuilder.js` — implementa el `ReportBuilder` con setters encadenables y validación en `build()`.
- **Modificado:** `backend/routes/reports.js` — la ruta `POST /api/reports` usa `ReportBuilder` antes de persistir en MongoDB.

Ventajas y limitaciones:

■ Ventajas:

- *Singleton*: gestión centralizada de la conexión, evita múltiples llamadas a `mongoose.connect()` y reduce fugas de conexiones.
- *Builder*: código más modular y legible; la validación y los valores por defecto viven en un único lugar, no en la ruta.

■ Limitaciones:

- El Singleton asume un proceso Node monolítico; en entornos con clustering cada proceso tendrá su propia instancia, lo cual es esperado pero conviene documentarlo.
- El Builder actual es sencillo; si aparecen múltiples variantes de `Report` podría combinarse con un *Factory* para producir distintas configuraciones.

Link GitHub: <https://github.com/AndreSanchezVega/PC2-DS>